University *of Ljubljana*
Faculty *of Computer and Information Science*

FRI ReferatBot – Domain-Specific assistant for UL FRI Student Affairs Office

Gašper Kolbezen, Samo Hribar and Gašper Suhadolnik

Abstract

General-purpose Large Language Models (LLMs) are highly capable at open-domain conversation, but they frequently hallucinate when asked institution-specific questions they were not trained on. This project addresses that limitation by building a domain-specific conversational assistant for the Student Affairs Office (slo., Referat) of the Faculty of Computer and Information Science at the University of Ljubljana. The assistant will serve the faculty's enrolled students by answering questions about enrolment procedures, exam regulations, thesis submission, student status, fees, and related administrative topics. We plan to implement a Retrieval-Augmented Generation (RAG) pipeline grounded in a curated corpus of UL FRI web pages, official study regulations, and FAQ documents. This report covers the project motivation, a survey of related work, an overview of our planned methodology, and a description of the proposed corpus.

Keywords

Retrieval-Augmented Generation, Large Language Models, Domain-Specific Chatbot, Student Affairs, Slovenian NLP

Advisors: Slavko Žitnik

Introduction

Students at university faculties routinely encounter administrative questions: How do I enrol in the next academic year? How many times can I take an exam? What happens if I miss the enrolment deadline? What documents do I need to submit my thesis? These questions are answered on the UL FRI website and in official study regulations, yet students frequently contact the Student Affairs Office (Referat) by email or in person for answers that are already publicly accessible. This imposes a significant workload on administrative staff and introduces delays for students.

General-purpose LLMs such as GPT-5 or Llama 4 cannot reliably answer these questions: UL FRI-specific rules (e.g., the precise ECTS thresholds for year advancement, the five-attempt exam policy, the Year Plus module for foreign students) are not part of their training data, and the models hallucinate plausible-sounding but incorrect answers. The goal of this project is to close that gap by building a specialised assistant that retrieves factual information from a verified, institution-specific knowledge base before generating a response.

Our target domain is the UL FRI Student Affairs Office. The closest existing analogue within the University of Ljubljana is the AI student assistant deployed on the Faculty of Mechanical Engineering (UL FS) website, which is embedded as a live chat widget under the title "AI pomoč za študente UL FS". Our system aims to replicate and extend this concept for UL FRI, with a systematic evaluation framework and reproducible code.

To create our dataset, we will gather data by scraping the UL FRI website (enrollment info, FAQs) and collecting UL rulebooks and administrative PDFs, which are accessible online. We will use Microsoft's MarkItDown to convert this data into Markdown for LLM parsing. To create an evaluation corpus, we will poll students to collect a set of real queries.

The core implementation will include a Retrieval-Augmented Generation (RAG) [1] architecture. We propose using the Slovenian GaMS model to process user intents and generate answers. For the retrieval pipeline, we will chunk our parsed Markdown documents, generate vector embeddings, and store them in PostgreSQL using the pgvector extension.

To control the model, we will use system prompting to enforce a "Student Affairs Advisor" persona. We will set guardrails so the system refuses to answer out-of-domain questions and handles unanswerable queries by directing the user to the student affairs office. Finally, we will evaluate the

system using automated retrieval metrics and human testing on our polled dataset.

Related Work

0.1 Domain-specific Knowledge Incorporation

Incorporating domain-specific knowledge into LLMs is done through the following main approaches: prompt engineering, retrieval-augmented generation (RAG) [1], and fine-tuning. Prompt engineering is the fastest and cheapest method, where carefully designed prompts provide the model with additional knowledge at runtime. While this technique is fast and quick to set up, it offers limited depth and accuracy in some cases, including domain tasks.

RAG [1] solves this issue by connecting the model to external knowledge sources, improving accuracy and transparency, though at the cost of added system complexity and latency. Knowledge is obtained from internal documents, websites, PDFs, company databases etc. and is typically stored in some vector database, such as FAISS, Pinecone, Weaviate, etc. During inference, relevant documents are first retrieved from the database based on user prompt using vector search, and are then passed on to the underlying LLM, which generates the answer.

Fine-tuning embeds domain expertise directly into the model, delivering high performance and consistency for specialized tasks, but requires significant data, time, and ongoing maintenance. It is also harder to update or add new knowledge, which is done by additional fine-tuning of the model.

In practice, these methods are often combined, with RAG [1] providing dynamic knowledge, fine-tuning ensuring consistent behavior, and prompting orchestrating interactions.

0.2 Existing Solutions

There are many existing university chatbot solutions using RAG architecture that differ in used technologies and purpose. One example of such system is a chatbot developed for Misissippi state university by Neupane et al., which uses BarkPlug v2 [2] architecture. For context retrieval it uses an embedding model for vectorization of external data sources and Chroma DB vector database. Actual context retrieval is done by retriever technique provided by LangChain [3] using Maximum Marginal Relevancy and Similarity Search methods. For the underlyign LLM, it utilizes OpenAi's gpt-3.5-turbo. The system achieved a mean RAGAS score of 0.96, and has proven itself to be practical and effective for real-world usage through various qualitative experiments.

Other examples of such chatbots are Meri AI [4] and AceIt-ECE-Capstone [5]. Meri AI [4] is a campus intelligence system, which runs a LangGraph workflow [3] for intent classification, route detection, context retrieval, and response generation. It uses Supabase PostgreSQL with pgvector as knowledge database containing content scraped from university resources. AceIt-ECE-Capstone [5] is a course assistant focused on study help. It uses RAG [1] pipeline where course data is embedded with Amazon Titan Text Embeddings v2 and stored in Amazon RDS PostgreSQL. It uses Llama 3.3 to generate responses based on user query and retrieved database knowledge.

Another example of university chatbot is a virtual AI assistant developed by fakulteta za strojništvo of University of Ljubljana. Unfortunately, it seems that there is no publicly available information about its architecture and even when asked the bot itself, it was unable to answer it accurately. It was, however, able to explain that the data used to provide user query with relevant context comes from UL FS webpage and standard Q&A entries.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[2] Subash Neupane, Elias Hossain, Jason Keith, Himanshu Tripathi, Farbod Ghiasi, Noorbakhsh Amiri Golilarz, Amin Amirlatifi, Sudip Mittal, and Shahram Rahimi. From questions to insightful answers: Building an informed chatbot for university resources. In *2024 IEEE Frontiers in Education Conference (FIE)*, pages 1–9. IEEE, 2024.

[3] langchain langchain. applications that can reason. powered by langchain. https://www.langchain.com/. Accessed: 2026-03-19.

[4] Meri ai: Ai-powered campus navigation & intelligence system adama science and technology university. https://github.com/CSEC-ASTU/Meri_AI. Accessed: 2026-03-19.

[5] Ace it - ai study assistant. https://github.com/UBC-CIC/AceIT-ECE-Capstone. Accessed: 2026-03-19.